COMP9242 System Documentation

Group 19
Han Hanh — z5452011
Luong Thang Nguyen — z5402969

December 2nd, 2025

Contents

Execution model **3**
Execution entities . . . 3

Timer driver **4**
Timer IDs allocation . . . 4
Data Structures . . . 4
Timer Registration . . . 4
Timer Removal . . . 4
Timer IRQ Callback . . . 4
Limitations . . . 5

System Call **5**
User-SOS Data Transfer . . . 5

Filesystem **5**
Assumptions . . . 5
Data structures . . . 6
Data Transferring . . . 6
From Asynchronous to Synchronous . . . 6
Operations . . . 6

Virtual memory manager **7**
Data Structure . . . 7
Frame Mapping & Unmapping . . . 7
Page Table Probing . . . 8
VM Regions . . . 8
Mapping On Fault . . . 8

Demand Paging **8**
Data Structures . . . 8
Pagefile Interactions . . . 9
Second-chance Page Replacement Policy . . . 9
Limitations . . . 9

Process Management **10**
Process creation . . . 10
Process destruction . . . 10
Process Waiting . . . 11
Concurrency & Synchronization model . . . 12

Testing **12**

Bonus Mark **12**
`sys_mmap()` and `sys_munmap()` . . . 12
`mmap()` . . . 13
`munmap()` . . . 13

Execution model

We use a thread-based model, specifically offloads SOS system call processing onto worker threads.

Execution entities

SOS main thread

The thread boots the OS that initializes core system components such as:

- IPC endpoint:
  - This helps other threads communicate with this main thread
  - We first save it in the global `ipc_ep` of all the worker threads so they can use it to communicate with this main thread, later we will "transfer" this ep to the interrupt thread so that they will communicate with interrupt thread instead.
- Notification object.
- Map the timer device.
- Network hardware.
- Register a handler function to be called when the network console receives incoming data.
- Page swap:
  - Initialize all the mutexes for data structures that are shared for virtual memory.
  - Open the pagefile via `nfs_open_async()`.
- Timer:
  - Start the internal counter (Timer E) with tick frequency of 1us.
- Register an IRQ handler for the timer.
- Initializes the `free_pids` queue.
- Create a pool of worker threads, up to the number of user processes supported (16 in our system).
- Create an interrupt thread.
- Create a bootstrap thread that will load the first user process.

Worker Threads

We have a fixed-size pool of 16 worker threads, each mapped to at most one user process at a time.

Each worker thread is assigned a PID when it starts handling system call for the assigned user process. All worker threads will enter the `syscall_loop()` to handle system calls and user process's faults.

Worker threads share the same VSpace (`seL4_CapInitThreadVSpace`) so that communications and data passing to each other is easier. However, we do ensure user processes have their own VSpace.

Interrupt Thread

We need to spawn an interrupt handler thread because it needs to run before the first process starts. Hence, we need to:

- Transfer the `ipc_ep` from the main thread to the interrupt thread.
- Unbind the notification from the main thread TCB, and bind it to the interrupt thread TCB.

As a result, interrupts are now delivered to the interrupt thread.

Worker thread will use this `ipc_ep` to communicate with the interrupt thread.

For asynchronous events, we attached the worker thread ID into the `thread_index` field of the arguments to the callbacks, so that when the interrupt thread receives and dispatches it, it can wake up the corresponding worker thread.

Bootstrap Thread

The bootstrap thread is basically a worker thread, but it needs to load the first user process. It will then enter the `syscall_loop()` when finished loading the first user process.

Timer driver

Timer IDs allocation

We use the `next_timer_id` global variable to determine the id of the next timer. `next_timer_id` is a `uint32_t`, initialized to 1 and wraps around when reached `UINT32_MAX`.

Data Structures

We keep track two sets of ids: `used_ids` and `removed_ids`. The reason for using a set is because it supports `O(log n)` insertion and look-up time. Hence, when calling `register_timer`, we can quickly determine the existence of a timer with the given id.

The `used_ids` set stores the ids that have been assigned to a timer. Whereas the `removed_ids` set saves the ids that have been called `remove_timer()` upon. The use cases of these sets will be clarified when it comes to the explanation of `remove_timer()` mechanism.

Moreover, we also keep track the nearest upcoming timer by using a min heap: `timeout_heap`. This min heap is sorted by the expected timeout timestamp of the timer (i.e. the timestamp when the timer finished). This is handy for implementing a tickless timer.

Timer Registration

The `register_timer()` method involves three phases: configuring `timeout_data`, adding it to `timeout_heap`, and reconfiguring the timer to wait on the earliest timestamp.

Firstly, `timeout_data` is a struct containing `delay` and `callback`. Since the hardware timer only accepts `delay` up to `UINT16_MAX`, we break the timer registration down into multiple, sequential calls to `register_timer` when `delay` exceeds `UINT16_MAX`. This is done by registering a timer whose `delay` is `UINT16_MAX` and `callback` is a call to:

```
register_timer(
    previous_delay - UINT16_MAX,
    original_callback,
    original_data
)
```

where `previous_delay` is the delay we used in the previous call to `register_timer()`. With this approach, we can support any amount of delay (within `uint64_t`, the type of `delay`), with respect to the hardware limit.

Secondly, we add this `timeout_data` to `timeout_heap`. And finally, we pop the top element from `timeout_heap`. This `timeout_data` popped from `timeout_heap` will have the smallest expected timeout timestamp, which we will then configure the hardware timer to wait on this.

Timer Removal

For timer removal, we first get the current earliest timeout by peeking at the `timeout_heap`.

If the earliest timeout is indeed the timeout we intend to remove, then we can safely pop it from the heap, and reconfigure the hardware timer to wait on the next earliest timeout.

If the earliest timeout is ***not*** the current earliest timeout, we perform a lazy deletion on this timer. This is because `timeout_heap` does not support random deletion and simulating it is ***inefficient*** in terms of time (e.g. popping all out and adding them back in will take `O(n * log(n))`). Instead, we add the timer id to the `removed_ids` set, which will then be encountered and skipped by the `timer_irq` callback. The next section will explain more on how the `timer_irq` callback works.

Timer IRQ Callback

The `timer_irq` callback is called when there's a recent timeout occurred. It's primary task is to invoke the registered callback associated with the recent timeout. However, we expand its functionality to do more than just that, which involves:

- Invokes registered callbacks for other timeouts that expire at the same time or even before. This is easily achieved because the `timeout_heap` already keeps the timeouts sorted by their expired timestamps.

- Skips the removed timers by checking the existence of their id in the `removed_ids` set. Then removes the id from the `removed_ids` and `used_ids` set so it can be reused later. This helps us perform lazy deletion on the removed timers.
- Reconfigure the hardware timer to the next earliest timeout.

Limitations

Our current timer driver can only register `UINT32_MAX` timers at the same period of time. This is because the `next_timer_id` is a `uint32_t`, so further calls to `register_timer` will be giving used ids. We can work around this by making `next_timer_id` to be a `uint64_t`, whose upperbound is practically unreachable (as in the number of timers at the same time).

Moreover, we do not have a nice way to handle `delay` that are too small, such as 1 microsecond, or `UINT32_MAX + 1` (as it would breakdown into two timeouts with `UINT32_MAX` and 1 as the delays, respectively). In fact, this can be handled by:

- Checking if `expected_timeout_timestamp - current_timestamp` is within a safe threshold. The challenge is to work out a threshold that allows the largest possible working set of delays while makes sure the driver can handle it fast enough.
- In case of large delays (greater than `UINT32_MAX`), makes sure the remaining delay time is larger than the aforementioned threshold. For example, when breaking down a delay equals to `UINT32_MAX + 1`, we should break it down into two timeouts with `(UINT32_MAX + 1) - (2 * threshold)` and `(2 * threshold)` as delays, respectively (in this case, we intentionally make the second timeout delay to be `(2 * threshold)` to ensure it's larger than `threshold`.

System Call

We leverage the initial code in `sos.c` formulate the communication protocols between SOS and its applications. Specifically, a system call is made by invoking the `seL4_Call(SOS_IPC_EP_CAP, tag)` function, where the `SOS_IPC_EP_CAP` is predefined, and `tag` includes the number of arguments.

The arguments data are sent to SOS via the IPC buffer, using `seL4_SetMR()` method. The first argument is always the system call number, which is defined in our custom library `libsossharedapi` to ensure consistency between SOS and its applications.

In SOS, we have implemented a system call dispatcher under `syscall_handler.c`, which invokes the appropriate system call based on the system call number. Each system call implementation is put under a their own file, namely `sys_<system_call_name>.c`, for better readability and navigation.

User-SOS Data Transfer

There are various system calls such as `sos_open`, `sos_write`, etc. that requires transferring data from user to SOS and vice-versa. These system calls will provide SOS with the user application's virtual address, so SOS can write/read data to/from.

The typical process for user-SOS data transferring is that we find the pages associated with the virtual address and write/read data to/from at the exact offset of the virtual address in that page.

In case that we could not find the associated frame, our OS has different behaviours for each of the below cases:

1. Copy data from user to SOS:
    - SOS will not perform dynamic frame allocation. Since the user's virtual address is not mapped, reading from it is the same as reading garbage data.
2. Write data from SOS to user:
    - SOS will perform dynamic frame allocation to the user's virtual address, if it lies within a valid virtual memory region. This makes sense because if the user's virtual address is valid, then we should map it and write data to it (similar to a VM fault handler).

Filesystem

Assumptions

- Our file system has a flat hierarchy so there are no directories.
- Number of files per process is 16.

- Open network console reader gives a file descriptor 0, as it acts as `stdin` in our system. We do **not** assume that `stdin` is already opened for a user process.
- `stdout` and `stderr` is assumed to be already opened.

Data structures

File Table

Open files are tracked using a per-process file table, with `PROCESS_MAX_FILES` entries. Since the file table is per-process, not global:

- Each process has its own private file descriptor space
- File descriptors from different processes do not collide.

File descriptors are mapped directly to the index of the process's file descriptor table array. The allocation policy is lowest-possible id, this gives us predictable behaviour and compatibility with standard POSIX semantics.

Each file table entry stores:

- A `mode` that the file was opened (`O_RDONLY`, `O_WRONLY`, or `O_RDWR`),
- A boolean flag `is_opened` indicating whether the file is currently opened,
- A `path` which is the file's path name,
- An NFS file handle `fh` received when open the file via `nfs_open_async()`.

Data Transferring

As mentioned in User-SOS Data Transfer section, we use the copyin-copyout method to transfer data from SOS to user applications and vice-versa.

From Asynchronous to Synchronous

The NFS library provides an asynchronous interface (and we must use it), while system calls in SOS must present a blocking, synchronous interface to the user processes.

To bridge this mismatch:

- We give the current worker thread ID to each `nfs_*_async()` call via the `private_data` argument.
- Each `nfs_*_async()` is followed by a `seL4_Wait()` on the current worker thread notification.
- The `nfs_*_callback()` will invokes `seL4_Signal()` on the notification with respect to the given thread ID in `private_data`.

In brief, I/O-related syscalls block their worker thread until the NFS callback completes. Note that this does not blocks the whole system, because there are other worker threads that are still running and able to perform I/O-related system calls as well.

Operations

To handle syscall related to file systems, the general flow is that we first extract any useful information that the client sends us via the message registers, perform the corresponding syscall handler, then reply, or copy data back to the user.

Open

- If the user opens the console with read access (`O_RDONLY`, `O_RDWR`) mode then file descriptor 0 is returned, we also have a mechanism to update that the system has one network console reader already via the global variable `nwcs_reader` which is set to the thread ID of the calling worker thread.
- If the user opens the console with just write access (`O_WRONLY`) mode then `CONSOLE_FD` (a hard-coded file descriptor for network console) is returned.

  Even though the user opens the same network console, we separate the modes into two different file descriptors to easier handle closing the network console reader. Hence, we can ensure that closing a **network console reader** (`fd == 0`) does not ***accidentally*** close the **network console writer** (`fd == CONSOLE_FD`) and vice-versa.

- Since POSIX semantics assume that `stdout` and `stderr` are already opened, the file table entry for `CONSOLE_FD` is initialized to `is_opened = true` and `mode = O_WRONLY`. In contrast, the client is expected

to open `stdin` manually, therefore, its entry (`fd == 0`) for each user process is initiallized to `is_opened = false` and `mode = O_RDONLY`.

Read & Write

We wanted to support reads and writes at any given number of bytes, however it is limited to 224KiB at most, due to `mmap` absent in SOS. To overcome this, we breaks the number of bytes into smaller chunks (as long as it does not exceed 224KiB). Hence, the typical workflow for handling read/write system calls is:

- Break down a given number of bytes into many chunks with max size of `BREAKDOWN_THRESHOLD` (which we chose to be 2^{16}),
- Malloc a buffer to read/write data from/to,
  - If writing to a file, copy data from user to buffer
- Fire an NFS async read/write call, block waiting until the callback finish,
- If reading from a file, copy data from buffer to user,
- Repeats until all bytes are sent/read.

It is worth mentioning that we use `nfs_read_async()` and `nfs_write_async()` instead of `nfs_pread_async()` and `nfs_pwrite_async()` because the former keeps track the offset in the NFS file handle already, so there's no need to save the current file offset into the PCBs.

Virtual memory manager

Data Structure

Page Table

Each process has their own pagetable and is accessible via the `page_global_directory` field of the PCB. The pagetable data structure follows a four-level model which includes:

- Page Global Directory (PGD): the top level paging structure
  - PCB keeps track of PGD.
  - Contains 512 entries of PUD.
- Page Upper Directory (PUD): the second level paging structure.
  - Contains 512 entries of PD
  - `ut_t *ut` and `seL4_CPtr slot` for book-keeping of the **page upper directory** capability.
- Page Directory (PD): the third level paging structure.
  - Contains 512 entries of PT
  - `ut_t *ut` and `seL4_CPtr slot` for book-keeping of the **page directory** capability.
- Page Table (PT): the fourth level paging structure.
  - Contains 512 entries of `page_metadata_t`
  - `ut_t *ut` and `seL4_CPtr slot` for book-keeping of the **page table** capability.

Page Metadata

We save a `page_metadata_t` in each entry of the page table. It refers to the SOS frames that are abstracted by the `frame_table` interfaces. A `page_metadata_t` includes:

- `frame_ref_t frame_ref`: A reference to the frame in the `frame_table`.
- `seL4_CPtr frame_cap`: A copied capability to the frame in the `frame_table`.

There are two more important fields that we later add into `page_metadata_t` to enable demand paging.

Frame Mapping & Unmapping

We implement our own version of `map_frame()` called `sos_shadow_map_frame()`. The word "shadow" comes from the fact that we need to keep track of the paging structures. Unlike `map_frame()`, our version creates the paging structure on demand and saves the `ut` pointer and seL4 capability associated with them.

On the other hand, we have `sos_shadow_unmap_frame()` which does the exact opposite: it unmaps the frame via `seL4_ARM_Page_Unmap()`, delete and free the capability, free the `ut` slot and free the frame back to `frame_table` via `free_frame()`.

Page Table Probing

`find_page()` is our primary function to find a `page_metadata_t` entry given a virtual address and an appropriate `page_global_directory` structure. By leveraging the four-level structure of the pagetable, we can perform:

- Constant time lookup by working out the appropriate index on each paging structure.
- Minimal levels traversed, as we return early if any paging structure does not exist.

VM Regions

We also introduce VM regions to each PCB so each process can keep track of the memory regions they possess. VM regions are stored as a linkedlist of `vm_region_t`, where each contains:

- `uintptr_t vaddr_base`: the starting virtual address of the region,
- `size_t size`: the size of the region,
- `seL4_CapRights_t rights`: the rights to read, write or both,
- `bool grows_downward`: `true` if the region this growing downwards (for example, the stack), `false` otherwise.

With VM regions, we can easily handle VM faults by looping through all regions and check if the fault address lies within any region and satisfies the region's rights.

The diagram below is a simple breakdown of the virtual memory layout in each user process. We acknowledge that there can be more sections before the Heap, but for simplicity we assume there are only two sections before it: Data+BSS and Text.

```
High Addresses
+---------------+
|   IPC Buffer  |
+---------------+
|     Stack     | <-- grows downward (STACK_TOP = 0x9000000000)
+---------------+
|  Mmap region  | <-- grows downward
+---------------+
|     Heap      | <-- grows upward
+---------------+
|    BSS/Data   |
+---------------+
|   Code/Text   |
+---------------+
Low Addresses
```

We can confirm that we can provide as much heap and stack memory as the address space can provide. This is because we create the paging structures on demand, hence we do **not** need to store all paging structures a pagetable can possibly have.

Mapping On Fault

Since we had VM fault working, we defer the actual mapping of pages until they are used. For example, we only allocate a few pages for the user process's stack initially, but it can grow as large as 300 pages as the VM fault handler maps it in dynamically.

Demand Paging

There are two goals to achieve in demand paging:

1. Evicting a page when the main memory and writing its content to pagefiles.
2. Swapping an evicted page back to main memory, restoring their original content.

A side quest for demand paging is to impose a page replacement policy, which will be discovered in the second-chance policy section.

Data Structures

To enable the aforementioned goals for demand paging, we introduced two key data structures:

- The `in_memory_pages` queue, and
- The `free_pagefile_offsets` queue.

The `in_memory_pages` queue stores all the pages that are currently in memory. The type of each element in this queue is `page_metadata_t`.

The `free_pagefile_offsets` queue stores all the offsets on the pagefile that is can be written to. It is handy when we try to evict a page and `free_pagefile_offsets` can quickly provide an available offset to write to the pagefile.

We have also added two additional data to the `page_metadata_t` struct:

- `int reference_bit`: to enable second-chance page replacement policy, will be discovered in the reference bit simulation section,
- `int pagefile_offset`: the location of this page content in the pagefile.

With these two fields, we can interact with the pagefile in a more precise and convenient way.

Pagefile Interactions

When reading or writing to the pagefile, the caller thread is blocked waiting for I/O operations on the NFS. However, it does not block other threads to interact with the pagefile.

Write To Pagefile

When a page is evicted, its content is written to the pagefile so it can be restored later. We first get an available offset by popping an element from the `free_pagefile_offsets` queue, then call `nfs_pwrite_async()` to write to the pagefile at that offset.

Note that we use `nfs_pwrite_async()` here because it allows us to write to the pagefile at any given offset.

Finally, we keep track the pagefile offset in the respective `page_metadata_t` structure so that we know where to **read from** the pagefile when swapping the page back to main memory.

Read From Pagefile

When we want to swap a page back to main memory, as mentioned above, we extract the pagefile offset associated with the `page_metadata_t` and use it to read from the pagefile via `nfs_pread_async()`.

Second-chance Page Replacement Policy

The most challenging part of implementing this page replacement policy is simulating the reference bit, as we don't have hardware support to achieve this.

Simulating The Reference Bit

Hence, as introduced above, we store a reference bit in each `page_metadata_t` and try to simulate it with the support of the VM fault handler.

Since we only know if a page is being accessed via a VM fault, we intentionally let a VM fault happens so that we can set the reference bit to 1. To achieve this, when we first map a virtual address to a frame:

- Initializes the `page_metadata_t` entry and creates the intermediate paging structures if needed. Makes sure to set the reference bit to 0.
- Do not use `seL4_ARM_Page_Map()`. This ensure that the next access to this page will cause a VM fault.
- When the page is caught in the VM fault handler, set the reference bit to 1 and **now** we can use `seL4_ARM_Page_Map()` to map this virtual address.

Now that we have a mechanism to simulate the reference bit, we can then implement the second-chance page replacement policy easily.

Limitations

We keep track all the free pagefile offsets in memory, which is not memory efficient and may ran out of memory if running the benchmark. We tend to work around this by increasing the size of the queue, but it is clearly not a solution to this problem. We have thought of saving a small amount of offsets in memory, and the rest can

be stored in a NFS file. This idea is quite similar to how the pagefile and the `in_memory_pages` queue work together.

Process Management

Process creation

Process IDs Allocation

There is one hard requirement that is process IDs must not be reused too soon. Our first solution was to put the available process IDs into a queue and save the time each enters the queue. When we need to get a new process IDs, only pop it if it's been staying in the queue **long enough** (compared to a threshold); otherwise, push it to the back of the queue. The problem with this solution is that an appropriate threshold is hard to define.

Our final solution is a bit different from this. We instead expand our user process pool to be twice as large as the number of user processes supported. For example, we current support exactly 16 user processes. Hence, the user process pool is actually 32 instead. This makes sure there is a high chance that the next available process ID has not been used recently.

ELF Loading

Since the ELF is entirely on NFS, we approach this by first reading the first 4KiB (assuming the header is less than 4KiB) to get the ELF headers. We then utilize `nfs_pread_async()` to read the ELF file on NFS at any offset provided by the header.

Extracting the `__vsyscall` Pointer During the initialization of the process stack, we need to set up a System V compliant task. One of the requirement is getting the virtual address of the `__vsyscall` section. However, the function SOS originally used were `elf_getSectionNamed()`, which assumes the `elf_t` passed in is a full ELF file. Hence, we tried reading the source code in `libelf` and figured out a way to overcome this.

Section Table We first need to load the section table into memory, as it is actually an array of section headers. The ELF header data provides the offset to the section header, so we can use `nfs_pread_async()` to read the section table and load it to memory.

String Table The string table is a centralized location for storing textual names. The ELF header data also provides an offset to the string table with respect to the ELF base.

Get Section Name This step is crucial to find the `__vsyscall` section. The `sh_name` field in a section header provides the offset to the section name, with respect to the location of the String Table.

Given that we have already got the string table offset (w.r.t ELF base) and the section name offset (w.r.t string table), we can deduce the offset of a section name, w.r.t ELF base:

$$section_name_offset = str_table_offset + section_header.sh_name$$

Given the section name offset w.r.t ELF base, we use `nfs_pread_async()` to retrieve the name and hence figure out which section header corresponds to `__vsyscall`.

Loading Rest of ELF We only needed to change the given `load_segment_into_vspace()` method a bit to get it work with the ELF on NFS. The key change is that instead of using `memcpy()` to copy data directly from the ELF file to a frame, we use `nfs_pread_async()` to copy data from the ELF file on NFS to a frame.

We decided to load the whole executable file at process startup to keep our VM fault handler logic simple. However, we can lazily load the executable file by marking the respective VM region as "originated from ELF", so that the VM fault handler knows when to read from the ELF on NFS.

Process destruction

Besides deallocating all the data structures we kept in the PCB, we also make sure to minimize undefined behaviours. This is achieve by two acts.

First, we call `seL4_TCB_Suspend()` on the current process TCB capability before destroying any data structures in the PCB. This is important because if we don't stop the current user process from executing, it might operate on deleted data structures / freed capability.

Second, after destroying all the data structures in the PCB, we suspend the worker thread that is assigned to the deleted user process and make it start running at the `syscall_loop()` again. We achieve this by:

- Suspending the worker thread using `seL4_TCB_Suspend()`.

  However, when a user process tries to delete itself, we do **not** suspend their assigned worker thread, because it's the one handling the `process_delete()` syscall.

- Using `seL4_TCB_WriteRegisters()` with appropriate `seL4_UserContext` to make the worker thread continue their execution at `syscall_loop()`.

The reason we do this is because the affected worker thread might be blocked waiting on an I/O operation or a process to exit, so it's sensible to early terminate it and have it ready to handle a new user process.

Process Waiting

We store a linkedlist of IPC endpoints, where each belongs to another worker thread, inside the PCB, called `waitlist`. Inside `process_wait()`, we add the worker thread's IPC endpoint to the PCB `waitlist`. The worker thread then blocks itself waiting for a signal from the user process response.

When a process exits, it goes through all the IPC endpoints and wakes up all the worker threads waiting on it.

Note that we do not use notifications to for signalling and waiting in this case. We instead uses IPC to wake up the worker threads because it allows us to send the message registers. Specifically, the signalling thread will make system call `SYSCALL_NULL_OPS` so that the waiting thread, in case it is forced to rerun at `syscall_loop()`, can receive the `SYSCALL_NULL_OPS` and ignore it gracefully.

The diagram below depicts the situation above:

```
+----------------------+
|Thread  ID| 0 | 1 | 2 |
|            v   v   v |
|Process ID| 0 | 1 | 2 | <- Process 0 waits on Process 1
|            |   ^     |
|            |   |     |
|             wait     |
+----------------------+
+----------------------+
|Thread  ID| 0 | 1 | 2 |
|            v   v   v |
|Process ID| 0 | 1 | 2 | <- Process 2 kills Process 0
|            ^       | |
|            |       | |
|            --kill--- |
+----------------------+
+----------------------+
|Thread  ID| 0 | 1 | 2 |
|                v   v |
|Process ID|   | 1 | 2 | <- Process 2 kills Process 1
|                ^   | |
|                |   | |
|                kill- |
+----------------------+
+----------------------+
|Thread  ID| 0 | 1 | 2 |
|            ^   |     |
|            |   |     |
|            NULLOPS   | <- Worker thread 1 make nullops syscall to worker thread 0
+----------------------+
```

If `seL4_Signal()` was made in the last step, our current `syscall_loop()` thought it was a syscall instead (due to the check `label == seL4_Fault_NullFault`) and handled it with garbage values in the message registers,

which caused undefined behaviours.

Concurrency & Synchronization model

Synchronization Primitive

We started with using a mutex as our main synchronization primitive. It is actually implemented by a notification and hardware support for atomic operations.

However, we quickly ran into an issue. There are nested function call paths that eventually lock the mutex twice, which causes a deadlock. We resolve this issue by using a recursive mutex, which allows locking itself multiple times within the execution of a thread.

Affected Data Structures

Below is a list of shared data structures that we apply the mutex on:

- `worker_threads` array in `threads.h`
- `user_processes` array in `user_process.h`
- `free_pids` in `user_process.h`
- `in_memory_pages` queue in `page_swap.h`
- `free_pagefile_offsets` queue `page_swap.h`
- `frame_table` in `frame_table.h`
- `nwcs_reader` in `sys_read.h`

Handling Outstanding Async Requests

There cases where a process already exits, but the asynchronous callbacks still fire. In these cases, we need to invalidate these callbacks and gracefully exit.

We achieve this by storing the `thread_id` and `expected_pid` field in the `private_data` in each call to `nfs_*_async()`, where:

- `thread_id` refers to the worker thread that issues the `nfs_*_async()`,
- `expected_pid` refers to the worker thread's assigned process. This value is similar to the `assigned_pid` field of the worker thread.

Note that when a user process exits, we set the `assigned_pid` field for the worker thread to be -1.

Hence, to invalidate such outstanding async requests, we check if the current `assigned_pid` field of the worker thread (associated with `thread_id`) matches the `expected_pid` in `private_data`. If it matches, then this means the user process is still alive and ready to receive a `seL4_Signal()` from the callback. Otherwise, the user process has already exited, and the callback should do no side-effects and return early.

Testing

We wrote some unit tests and put it under the `projects/aos/apps/tests` folder. Whenever we need to test our system, we switch the `APP_NAME` (located in `user_app.h`) to `tests` and start the system normally. We also did some colourings on the text to signal passed tests.

The tests includes:

- `vm.c` running virtual memory tests. We used the demo code from one from Milestone 3 and also wrote a thrash test which resembles the benchmark code.
- `process.c` running process related tests.
- `filesystem.c` running file system tests. These involves testing on open, read, and write operation on the files on NFS.

Bonus Mark

`sys_mmap()` and `sys_munmap()`

Data Structure: Red-Black Tree

Each user process maintains an RB-tree (`mmap_tree`) in the `mmap_region` field, indexed by the base virtual address of the region. Each node stores:

- `vaddr_base`: starting virtual address of the region
- `length`: size of the mapping, page-aligned
- `prot`: protection flags

The RB-tree provides:

- `O(log n)` lookup for containing region, given the virtual address.
- `O(log n)` insertion of a new `mmap` region.
- An ordered view of all `mmap` regions.

`mmap()`

We choose to perform automatic address selection: the given `addr` argument of `mmap()` is ignored and treated purely as a hint.

To avoid overlap with the stack, the `mmap` area grows downward, beginning at **two pages** below the stack's guard page.

Every successful `mmap()` call allocates one contiguous region, which is then inserted into the red-black tree.

To assign an address for a new mapping:

1. Traverse the `mmap_tree` in ascending order of the base virtual address
2. Inspect the gap between each pair of consecutive regions
3. If a gap is large enough for the requested length (page-aligned), return the base of that gap.
4. If no gap is suitable, return `-ENOMEM`.

Gap searching runs in O(n), where `n` is the number of `mmap` regions.

We have only added the region into the PCB's `mmap_regions`. The actual mapping is handled by the VM fault handler, where it makes sure to look up `mmap_regions` to verify the validity of the fault address.

`munmap()`

Given the current scope of the project, `munmap()` is only required to support `free()`, so:

- the supplied `addr` must be page-aligned
- the supplied `length` must exactly match the allocated mapping

Thus, we can directly identify the region via a lookup (`O(log n)`) in the RB-tree using `vaddr_base`.

Once the region is found, we deallocates it page-by-page via the `destroy_page()` function which we created for deleteing `user_process`.

Finally, the region is removed from the PCB's `mmap_regions` in `O(log n)` time.